

Code review — Variante BAC

Runde de review pe variantele tale de bac, cea mai nouă sus.

Runda 1 — 2026-09-15 — sha 19f5193

Branch `master`, curat, nimic necomis. 4 fișiere de variante, 500 de linii.

Poarta de compilare: `main.cpp` compilează (dar nu rulează nimic — vezi M4). `temaVar3.h` și `temaVar4.h` **NU compilează**. Verificat cu `g++ -std=c++17 -fsyntax-only`.

Ce am verificat prin rulare, nu din ochi: toate cele 4 exerciții de recursivitate din Subiectul III, exercițiul cu vocalele, matricea din varianta 1, și ultimul cuvânt generat prin backtracking (prin generare exhaustivă, în Python).

Întâi, ce ai făcut bine

Nu e politețe, sunt lucruri verificate:

Ce	Rezultat
V1 SUB3 — <code>f(121,1)</code>	ai scris <code>12345</code> · rulat: 12345 ✓
V2 SUB3 — <code>f(12345)</code>	ai scris <code>5310135</code> · rulat: 5310135 ✓
V3 SUB3 — <code>f(26)</code>	ai scris <code>xyyy</code> · rulat: xyyy ✓
V1 ex.5 — matricea <code>min(i,j)</code>	rulat cu n=4, m=5: exact matricea din enunț ✓
V1 ex.3 — mijlocul segmentului	corect
V3 — <code>cif(a,b)</code>	corect
V2 item 1, V3 item 1, V4 item 1, V4 item 2	răspunsuri corecte

Trei din patru urmăririi de recursivitate, corecte. Aia e partea grea din Subiectul III și o ai. Problemele de mai jos sunt de altă natură — și una singură ține de gândire.

● Critice

B1 — `temaVar3.h:63,67` și `temaVar4.h:82,87` : codul nu compilează

CUM E ACUM

```
cout<<'x'; | printf("%c",'x');
```

```
./temaVar3.h:63:15: error: unexpected character <U+2019>
./temaVar4.h:82:3: error: use of undeclared identifier 'cout'
```

DE CE

Sunt trei lucruri suprapuse, și fiecare te-ar fi picat separat.

Primul: bara verticală `|` din foaia de examen **nu e cod**. E notația prin care subiectul îți arată aceeași instrucțiune în două limbaje — `cout<<...` pentru C++, `printf(...)` pentru C. Alegi una și o scrii pe ea. Copiată integral, `|` devine operatorul „sau pe biți” și compilatorul se împiedică în el.

Al doilea: ghilimelele. În PDF-ul de bac apar ca `'` și `"` (ghilimele tipografice). În C++, caracterele se scriu cu `'` și `"` — apostrof și ghilimea drepte, de pe tastatură. Arată aproape identic pe ecran, dar pentru compilator sunt caractere complet diferite, iar mesajul pe care ți-l dă (`unexpected character <U+2019>`) e exact despre asta.

Al treilea, și cel mai important: **nu ai aflat niciodată că e stricat**. `main.cpp` include doar `varianta-1.h`. Celelalte trei fișiere nu sunt incluse nicăieri, deci compilatorul nu se uită în ele niciodată. Un fișier care nu e inclus e un fișier necontrolat — poate conține orice. La bac, subiectul III se scrie pe hârtie și nu te salvează nimeni; în laborator, singura ta plasă de siguranță e compilarea, și tu ai dezactivat-o fără să vrei.

FIX

```
// temaVar3.h:63
cout << 'x';
// temaVar3.h:67
cout << 'y';
```

Iar în `temaVar4.h`, care începe direct cu garda de include, adaugă la început:

```
#include <iostream>
using namespace std;
```

Apoi include toate cele patru fișiere în `main.cpp`, chiar dacă nu apelezi nimic din ele. Dacă compilează, știi că există.

B2 — `temaVar2.h:61-71` : `rezolvare5` nu rezolvă cerința

CUM E ACUM

```
for (int i = 0; i < strlen(cuvant); i++) {
    if (esteVocala(cuvant[i])) {
        cout << cuvant[i];
    }
}
```

Rulat pe `bacalaureat` :

```
aaauea          ← ce dă codul tău
baAcaAlaAuUreEaAt ← ce cere enunțul
```

DE CE

Cerința are două jumătăți, iar codul tău nu o face pe niciuna complet.

Prima: **consoanele trebuie să rămână**. Enunțul spune „restul literelor nemodificându-se”. Tu ai scris doar ramura `if (vocală)`, fără `else`, deci fiecare consoană e sărită tăcut. Din `bacalaureat` rămân doar vocalele.

A doua: **vocala se dublează cu majusculă**. `a` devine `aA`, `e` devine `eE`. Tu afișezi vocala o singură dată, deci chiar și vocalele sunt greșite.

Rezultatul e o funcție care rulează, nu dă eroare, și afișează ceva plauzibil — genul de greșeală care la bac îți ia toate cele 10 puncte, pentru că n-ai de unde să-ți dai seama fără să compari cu exemplul din enunț. De aia enunțurile de bac **au mereu un exemplu**: nu e decor, e testul tău.

FIX

```
void rezolvare5(){
    char cuvant[21];
    cin >> cuvant;
    int n = strlen(cuvant);
    for (int i = 0; i < n; i++) {
        cout << cuvant[i];
        if (esteVocala(cuvant[i]))
            cout << (char)(cuvant[i] - 'a' + 'A');
    }
}
```

`cuvant[i] - 'a' + 'A'` e trecerea de la literă mică la literă mare: scazi poziția lui `a`, aduni poziția lui `A`. Mai scurt, ai și `toupper(cuvant[i])` din `<cctype>`.

B3 — **varianta-1.h:10** : raționamentul e bun, litera e greșită

CUM E ACUM

- a. `(x<8) && (x>=5)` b. `(x<=8) || (x>5)`
c. `(x>8) || (x<=5)` d. `(x<=8) && (x>5)`

`x>5 si x<=8` raspuns B

DE CE

Ai scris condiția corect: `x>5 si x<=8`. E exact intervalul `(5,8]`. Apoi ai bifat **b**, care e `(x<=8) || (x>5)` — un **sau**, nu un **și**. Condiția pe care ai gândit-o e la litera **d**.

Merită să te uiți o secundă la ce înseamnă b: fiind un **sau** între „mai mic sau egal cu 8” și „mai mare ca 5”, e adevărată pentru orice număr real. Pentru $x=1000$ e adevărată ($1000>5$). Pentru $x=-1000$ e adevărată ($-1000<=8$). Un răspuns care e mereu 1 nu poate caracteriza un interval.

E o greșeală de transcriere, nu de gândire — și exact de aia e periculoasă. Te costă 4 puncte pentru un răspuns pe care îl știai. **După ce alegi litera, recitește expresia de la litera aia și compar-o cu ce ai scris tu.** Zece secunde.

FIX

raspuns d

B4 — șapte cerințe fără rezolvare

Nu sunt greșite. Nu există.

Fișier	Cerința	Stare
<code>varianta-1.h:137</code>	SUB3 pb.3 — numere din <code>bac.txt</code> divizibile cu n	doar enunțul
<code>temaVar2.h:81</code>	SUB3 item 1 — ultimul cuvânt generat (backtracking)	fără răspuns
<code>temaVar3.h:92</code>	pb.4 b) — cel mai mare palindrom din cifrele lui n	doar <code>cif</code> , partea b) lipsește
<code>temaVar4.h:38</code>	item 4 — eliminarea literelor mici din șir	doar enunțul
<code>temaVar4.h:53</code>	item 5 — matricea cu diagonala secundară	doar enunțul
<code>temaVar4.h:90</code>	SUB3 item 2 — <code>f(15,2)</code>	tabel început, fără răspuns final
<code>temaVar4.h:102</code>	SUB3 pb.3 — numere cu cel mult 2 cifre din <code>NR.TXT</code>	doar enunțul

Două dintre ele au răspuns scurt și ți-l dau acum, ca să vezi dacă tabelul tău ducea acolo:

`temaVar4.h` — `f(15,2)` afișează `01111`. (Rulat.) Tabelul tău se oprea la `f(15,17)`, care e fix locul unde începe partea interesantă: acolo se afișează `0`, și abia apoi, la întoarcerea din recursivitate, cele patru `1`. Ai oprit urmărirea exact înainte de a se afișa ceva.

`temaVar2.h` — ultimul cuvânt generat e `eded`, litera `d`. (Verificat prin generarea tuturor celor 4 litere, în ordine.) Logica: vrei cuvântul cel mai mare, deci pornești cu `e`; a doua literă nu poate fi vocală, deci cea mai mare consoană, `d`; a treia poate fi orice, deci `e`; a patra nu poate fi vocală, deci `d`. Cursa e între `eded` și `edde` — câștigă `eded`, pentru că pe poziția a treia `e` bate `d`.

Importante

M1 — `temaVar4.h:1`: fișierul n-are `#include <iostream>` și nici `using namespace std;`. Celelalte trei le au. Chiar după ce repari B1, fișierul tot nu compilează singur.

M2 — `varianta-1.h:84` : `int A[100][100]` pe stivă = 40.000 de octeți. Enunțul garantează `n, m ≤ 10`, deci îți trebuie 10×10. Merge azi, dar e obiceiul din care apar stack overflow-urile când crește dimensiunea. Declară `int A[11][11]` și indexează de la 1, cum faci deja.

M3 — `temaVar2.h:66` : `strlen(cuvant)` în condiția buclei. Se recalculează la fiecare pas: parcurge șirul de la capăt de fiecare dată. Pentru 20 de caractere nu contează, dar e un reflex care te costă când șirul are 100.000. Calculează o dată, înainte de buclă — vezi fix-ul de la B2.

M4 — `main.cpp:7-9` : tot ce apelează ceva e comentat. Programul compilează și nu face nimic. Efectul secundar e B1: fără apeluri, fără includeri, fără verificare. Un `main` care nu rulează nicio soluție nu e o poartă, e o decorație.

M5 — `varianta-1.h` n-are gardă de includere. Celelalte trei au `#ifndef ... #define ... #endif`. Acum merge fiindcă e inclus o singură dată; în momentul în care incluzi toate cele patru fișiere (fix-ul de la B1), lipsa gării devine eroare de redefinire.

Cleanup

C1 — `varianta-1.h:63-71` : `minim` are o ramură în plus. `if(a<b) return a; if(b<a) return b; return a;` — a doua verificare e inutilă, cazul „egale” cade oricum pe ultimul `return`. Scurt: `return a < b ? a : b;`

C2 — `bac1.txt` vs `bac.txt`. Enunțul cere fișierul `bac.txt`. Al tău se numește `bac1.txt`. Când ajungi să rezolvi cerința, programul n-o să-l găsească și o să pară că nu merge citirea.

C4 — executabilul și fișierele obiect sunt comise în repo. `bin/Debug/VARIANTE-BAC.exe` și `obj/Debug/main.o` sunt urmărite de git, iar `.gitignore` nu există. Sunt fișiere pe care le regenerează compilatorul la fiecare build — în repo n-au ce căuta: se schimbă la fiecare rulare, umflă istoricul și dau conflicte fără rost. Adaugă un `.gitignore` cu `bin/` și `obj/`, apoi scoate-le din urmărire cu `git rm -r --cached bin obj`.

C3 — `varianta-1.h:86-90` : citești `m` înainte de `n`. Enunțul le dă în ordinea `n`, apoi `m`. Nu e greșit, dar la bac se punctează după enunț.

Before / After — B2

```
// ÎNAINTE – afișează "aaauea" pentru "bacalaureat"
for (int i = 0; i < strlen(cuvant); i++) {
    if (esteVocala(cuvant[i])) {
        cout << cuvant[i];
    }
}
```

```
// DUPĂ – afișează "baAcaAlaAuUreEaAt"
int n = strlen(cuvant);
for (int i = 0; i < n; i++) {
    cout << cuvant[i];
    if (esteVocala(cuvant[i]))
        cout << (char)(cuvant[i] - 'a' + 'A');
}
```

Q&A — trei întrebări

1. În `temaVar3.h` ai scris `cout<<'x'; | printf("%c",'x');`. De ce crezi că nu ți-a semnalat nimeni greșeala asta până acum, deși fișierul e în proiect de trei commit-uri?
2. La `f(15,2)` tabelul tău se oprește la `f(15,17)`. Ce se întâmplă în funcție când `x>n`, și de ce cifrele care se afișează după momentul ăla apar în ordine inversă față de ordinea apelurilor?
3. La varianta 1 ai scris corect condiția `x>5 si x<=8`, dar ai bifat litera b. la expresia de la b — `(x<=8) || (x>5)` — și găsește un singur număr real pentru care e **falsă**. Cât timp ți-a luat?

Ce urmează

Ordinea în care aș lucra:

1. **B1** — repară cele patru linii, include toate fișierele în `main.cpp`, compilează. 10 minute, și abia după asta știi pe ce stai.
2. **B3** — corectează litera. 10 secunde.
3. **B2** — rescrie `rezolvare5` și **testează-o pe bacalaureat**, comparând cu exemplul din enunț.
4. **B4** — cele cinci cerințe nerezolvate. Începe cu `temaVar4.h` item 5 (matricea cu diagonala secundară): seamănă cu matricea din varianta 1, pe care deja ai rezolvat-o corect.

Ce e de reținut din runda asta: din cele patru probleme critice, **una singură ține de gândire** (B2). Celelalte trei sunt lucruri pe care le știai și le-ai pierdut pe drum — o literă bifată greșit, o bară copiată din PDF, o urmărire oprită cu un pas înainte de final. La bac, ăsta e tipul de punctaj care se pierde cel mai ușor și se recuperează cel mai ieftin.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro